

## Quiz: Indexes (2)

### Collaboration policy:

You may NOT:

1. discuss the exam with any human other than Mike; this includes:
  - (a) asking your friend for clarification about what a problem is asking
  - (b) asking your friend if they've finished working on the quiz
  - (c) posting questions to github, stackoverflow, or any other online forum

You may:

1. take as much time as needed
2. use any written notes / electronic resources you would like (including postgres documentation and AI systems like ChatGPT)
3. use the lambda server and connect to a running postgres instance
4. ask Mike to clarify questions via email

Name:

---

All of the problems assume the following schema.

```
CREATE TABLE users (  
    id_users BIGINT PRIMARY KEY,  
    created_at TIMESTAMPTZ,  
    username TEXT NOT NULL,  
    description TEXT  
);  
  
CREATE TABLE tweets (  
    id_tweets BIGINT UNIQUE NOT NULL,  
    id_users BIGINT REFERENCES users(id_users),  
    in_reply_to_user_id BIGINT REFERENCES users(id_users),  
    created_at TIMESTAMPTZ,  
    text TEXT  
);  
  
CREATE TABLE tweet_tags (  
    id_tweets BIGINT REFERENCES tweets(id_tweets),  
    tag TEXT,  
    PRIMARY KEY(id_tweets, tag)  
);
```

1. Recall that certain constraints create indexes on the appropriate columns. List the equivalent `CREATE INDEX` commands that are run by these constraints.

**Solution:**

```
CREATE INDEX ON users(id_users);  
CREATE INDEX ON tweets(id_tweets);  
CREATE INDEX ON tweet_tags(id_tweets, tag)
```

Any variation of the above resulted in missing all points.

**Note:** When solving all subsequent problems, assume only the indexes above have been created. For example, if problem 3 asks you to create an index, you should not assume that index has been created when working on problem 4.

2. Create index(es) so that the following query will run as efficiently as possible.

Do not create any unneeded indexes; if no new indexes are needed, say so.

```
SELECT count(*)
FROM users
WHERE (id_users >= :min_id_users)
      OR (username >= :min_username AND username < :max_username)
```

**Solution:**

```
-- CREATE INDEX ON users(id_users); -- not needed due to unique c
CREATE INDEX ON users(username);
```

The best scanning strategy possible is a bitmap scan. It is impossible with a btree to efficiently combine the results of OR commands together.

3. Create index(es) so that the following query will run as efficiently as possible.

Do not create any unneeded indexes; if no new indexes are needed, say so.

```
SELECT count(*)
FROM users
WHERE username=lower(:username);
```

**Solution:**

```
CREATE INDEX ON users(username);
```

The index must exactly match one side of the operator. In this case the following index does not match either side of the operator and is incorrect:

```
CREATE INDEX ON users(lower(username));
```

4. The following SQL query returns tweets where either the text or the description of the user match a FTS query.

```
SELECT id_tweets
FROM tweets, users
WHERE tweets.id_users == users.id_users
      AND ( to_tsvector('english', text)
            || to_tsvector('english', description)
            )
            @@ to_tsquery('english', :query)
;
```

Notice that the WHERE clause contains two operators:

1. The equality operator can be sped up using standard btree indexes.
2. The @@ operator, however, cannot be sped up as written because it references columns from two separate tables, and we cannot create an index on two separate tables.

Rewrite the query above into an equivalent query where all operators in the query can be sped up using indexes. Also provide any needed indexes.

**Solution:** Answer to the original question (without the == operator):

The key idea is to “factor out” the || operator so that we have multiple conditions, each mentioning only a single table:

```
SELECT id_tweets
FROM tweets, users
WHERE (to_tsvector('english', text)
      @@ to_tsquery('english', :query)
      )
      OR ( to_tsvector('english', description)
          @@ to_tsquery('english', :query)
          )
;
```

Then we can build gin indexes like so:

```
CREATE INDEX ON tweets USING GIN(
    to_tsvector('english', text)
);
CREATE INDEX ON users USING GIN(
    to_tsvector('english', description)
);
```

Answer to the updated question (as written above with the == operator):

The key idea is to “factor out” the || operator so that we have multiple conditions, each mentioning only a single table:

```
SELECT id_tweets
FROM tweets
WHERE (to_tsvector('english', text)
      @@ to_tsquery('english', :query)
      )

UNION ALL

SELECT id_tweets
FROM tweets
JOIN users USING (id_users)
WHERE ( to_tsvector('english', description)
      @@ to_tsquery('english', :query)
      )
;
```

The following index speeds up the first GIN query maximally:

```
CREATE INDEX ON tweets USING GIN(
    to_tsvector('english', text)
);
```

Speeding up the second SELECT is a bit more complex because it involves both a JOIN and a WHERE. I would create the following two indexes:

```
CREATE INDEX ON users USING GIN(
    to_tsvector('english', description)
);
CREATE INDEX ON tweets (id_users);
```